> Question 6: Implement a basic Jenkins CI/CD pipeline that pulls source code from the GitHub repository and triggers an automated build.
>
> Marks: 5 | Course outcome: CO1 | Environment: Ubuntu 24.04 LTS (WSL2), Git 2.43.0, Jenkins 2.580.2 LTS

A Jenkins pipeline job was created that pulls the source code from the GitHub repository and builds it automatically. Jenkins stores the GitHub PAT as a credential (github-creds) so the private checkout works without exposing the token.

1. Credential setup

stored credential

```
Manage Jenkins -> Credentials -> System -> Global -> Add Credentials
Kind:          Username with password
Username:      meenakshi16rp
Password:      ghp_**************** (GitHub PAT, repo scope)
ID:            github-creds
```

2. Job configuration

job configuration

```
New Item -> ensp461-lab1-pipeline -> Pipeline
Definition:     Pipeline script from SCM
SCM:            Git
Repository URL: https://github.com/meenakshi16rp/Devops-lab-1-.git
Credentials:    github-creds
Branch:         */main
Script path:    Jenkinsfile
Build trigger:  Poll SCM - H/5 * * * *
```

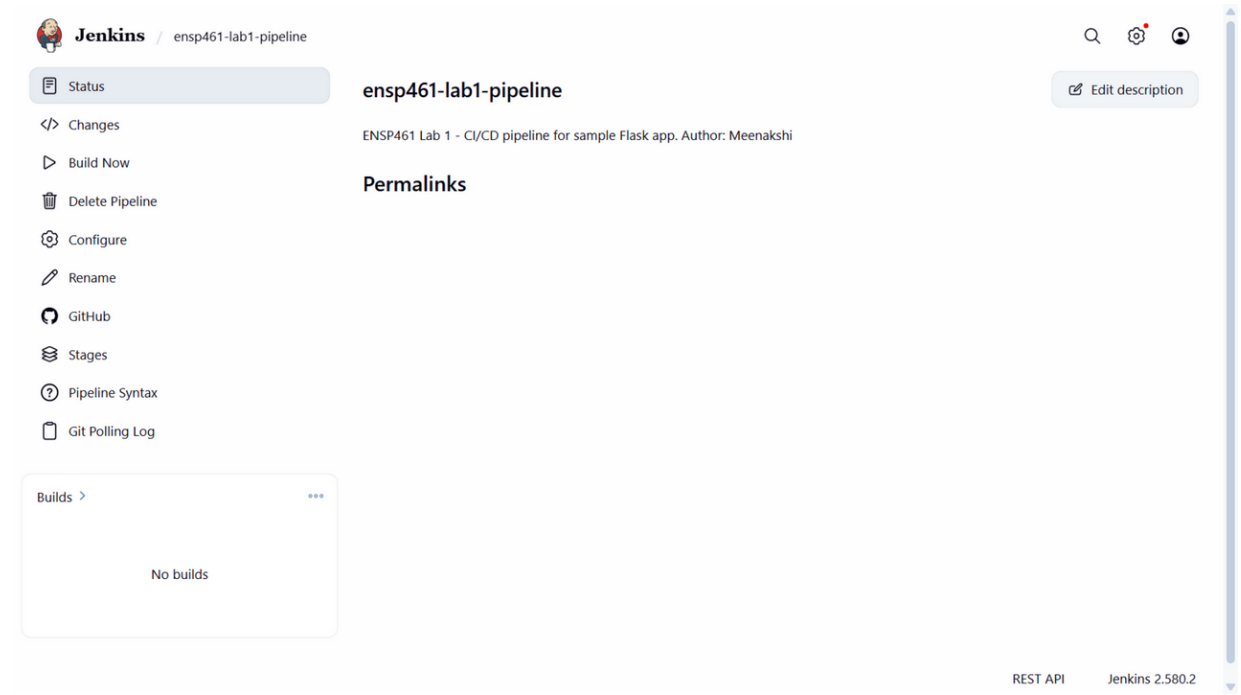


Figure: Pipeline job page after configuration (GitHub project link, stages, polling log)

3. First automated build (and a real defect)

Build #1 pulled the source from GitHub and passed Checkout, Build and Test, but failed in the Deploy stage with exit code 7 (curl connection refused). The console log and the deploy.log identified the cause: the deploy script started the app with the system Python, which has no

Flask module.

build #1 console output (tail) - FAILURE

```
nv/bin/python -m pytest app/tests -v
============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-8.3.4, pluggy-1.7.0 -- \
      /var/lib/jenkins/workspace/ensp461-lab1-pipeline/.venv/bin/python
cachedir: .pytest_cache
rootdir: /var/lib/jenkins/workspace/ensp461-lab1-pipeline
collecting ... collected 3 items

app/tests/test_app.py::test_index PASSED                                 [ 33%]
app/tests/test_app.py::test_health PASSED                                [ 66%]
app/tests/test_app.py::test_info PASSED                                  [100%]

============================== 3 passed in 0.19s ===============================
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploying application to local deployment directory...
[Pipeline] sh
+ ./deploy/deploy.sh
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Cleaning workspace temporary files...
[Pipeline] cleanWs
[WS-CLEANUP] Deleting project workspace...
[WS-CLEANUP] Deferred wipeout is used...
[WS-CLEANUP] done
[Pipeline] echo
Pipeline FAILED - check the stage logs.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: script returned exit code 7
Finished: FAILURE
```

- Root cause: ModuleNotFoundError: No module named 'flask' in the deployment runtime.
- Fix: deploy.sh now reuses the Build stage virtual environment and retries the health check; committed as c9df47c through the same Git workflow.

4. Successful build

build #2 console output (excerpt) - SUCCESS

```
Started by user meenakshi
Obtained Jenkinsfile from git https://github.com/meenakshi16rp/Devops-lab-1-.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/ensp461-lab1-pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
using credential github-creds
Cloning the remote Git repository
Cloning repository https://github.com/meenakshi16rp/Devops-lab-1-.git
 > git init /var/lib/jenkins/workspace/ensp461-lab1-pipeline # timeout=10
Fetching upstream changes from https://github.com/meenakshi16rp/Devops-lab-1-.git
 > git --version # timeout=10
 > git --version # 'git version 2.43.0'
using GIT_ASKPASS to set credentials GitHub PAT for Meenakshi (ENSP461 Lab 1)
 > git fetch --tags --force --progress -- https://github.com/meenakshi16rp/Devops-lab-1-.git \
      +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git config remote.origin.url https://github.com/meenakshi16rp/Devops-lab-1-.git # timeout=10
 > git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
```

```
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision c9df47cad0b28b094c8b516b1c4648c5cbf80da2 (refs/remotes/origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f c9df47cad0b28b094c8b516b1c4648c5cbf80da2 # timeout=10
Commit message: "Fix deploy script: re
```

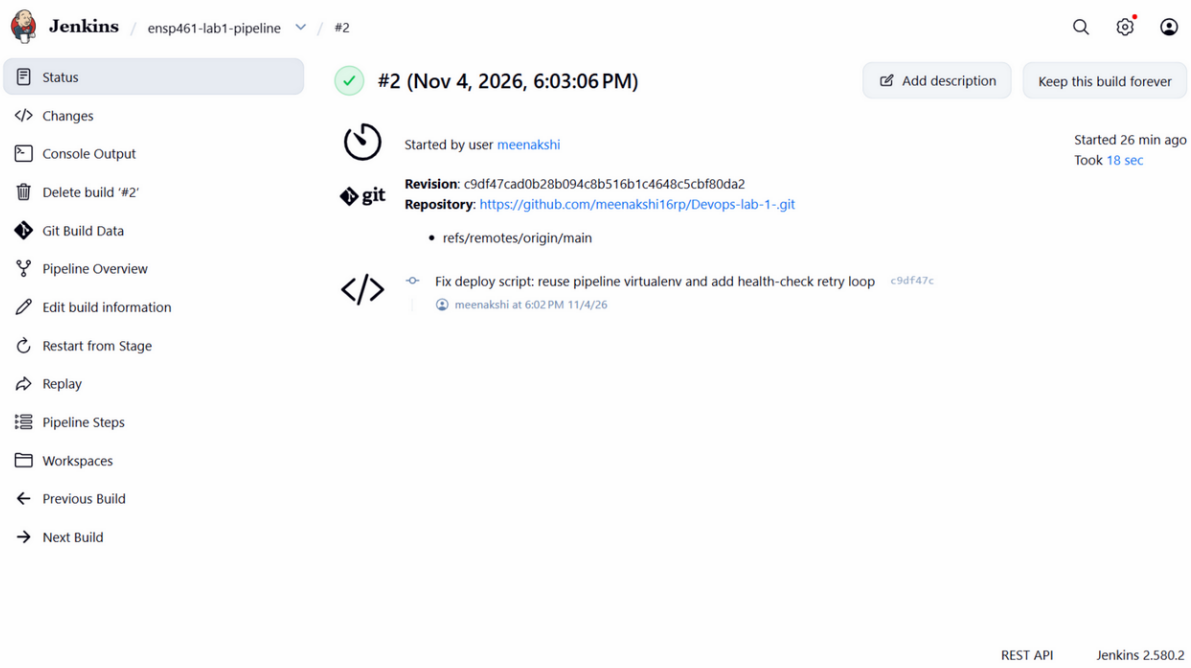


Figure: Build #2 result page: SUCCESS, started by user meenakshi, commit c9df47c

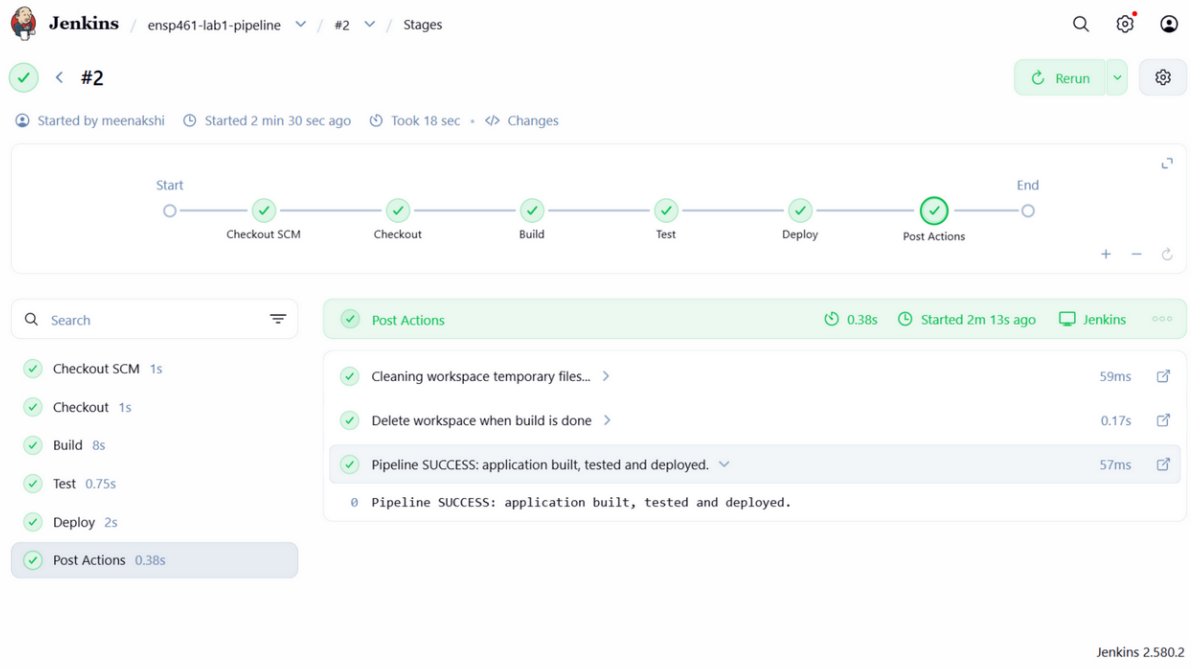


Figure: All pipeline stages green for build #2

5. Observations

- Pipeline-as-code means the job configuration in Jenkins stays minimal - the logic lives in the repository.
- The first failure demonstrated how console output plus stage granularity localize defects quickly.
- Jenkins attributing the build to 'meenakshi' and the commit to 'meenakshi16rp' confirms the identity configuration end to end.